

RAPPORT DE PROJET DE FIN D'ÉTUDES

CinéApp Android

Écosystème Intelligent de Streaming Cinématographique

Filière : Génie Informatique
Année universitaire : 2025– 2026

réalisé par : LAARIBAT HAITAM

Dédicaces

*À nos familles,
pour leur soutien indéfectible et leur amour sans limite tout au long de ce parcours.*

*À mon encadrant,
pour leurs précieux conseils, leur disponibilité et leur guidance tout au long de ce projet.*

À tous ceux qui croient en la force de l'innovation technologique.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Mes sincères remerciements s'adressent tout d'abord à mon encadrant pédagogique, pour ses conseils éclairés, sa disponibilité constante et ses orientations pertinentes qui ont guidé mon travail tout au long de ce projet.

Je remercie également l'ensemble du corps enseignant de mon établissement pour la formation de qualité dispensée tout au long de mon cursus académique, ainsi que le jury qui me fait l'honneur d'évaluer ce travail.

Mes remerciements vont aussi à mes collègues et amis pour l'ambiance de travail stimulante, les échanges fructueux et le soutien moral apporté dans les moments difficiles.

Enfin, je remercie ma famille pour sa patience, son soutien et ses encouragements permanents qui ont été une source inépuisable de motivation.

Résumé

Ce projet de porte sur la conception et le développement d'un écosystème mobile complet dédié au cinéma, baptisé CinéApp. L'application Android, développée en Java natif, offre une expérience cinématographique enrichie en combinant plusieurs technologies de pointe.

L'architecture client/serveur repose sur un frontend Android utilisant Volley pour les requêtes réseau et Glide pour la gestion des images, couplé à un backend Node.js (Express) assurant l'authentification des utilisateurs. L'intégration de l'API TMDB permet d'accéder à une base de données cinématographique riche et actualisée (films populaires, bandes-annonces, métadonnées), tandis que l'API Groq (modèle LLaMA 3.3) apporte une couche d'intelligence artificielle pour l'analyse intelligente des films.

Le projet se distingue par cinq fonctionnalités avancées : la vision par ordinateur via Google ML Kit pour la détection de visages et l'application de filtres dynamiques, la géolocalisation des cinémas à proximité via OSMDroid (OpenStreetMap), la lecture de bandes-annonces YouTube, la capture de photos de profil, et un système de recommandation IA personnalisé.

Mots clés

Android • Node.js • TMDB API • Intelligence Artificielle • Groq API • LLaMA 3.3 • Google ML Kit • OSMDroid • Vision par Ordinateur • Géolocalisation • Architecture Client/Serveur

Abstract

This final year project focuses on the design and development of a comprehensive mobile ecosystem dedicated to cinema, called CinéApp. The Android application, developed in native Java, offers an enriched cinematic experience by combining several cutting-edge technologies.

The client/server architecture relies on an Android frontend using Volley for network requests and Glide for image management, coupled with a Node.js (Express) backend handling user authentication. Integration of the TMDb API provides access to a rich and up-to-date cinematographic database (popular movies, trailers, metadata), while the Groq API (LLaMA 3.3 model) adds an artificial intelligence layer for intelligent film analysis.

The project stands out through five advanced features: computer vision via Google ML Kit for face detection and dynamic filter application, nearby cinema geolocation via OSMDroid (OpenStreetMap), YouTube trailer playback, profile photo capture, and a personalized AI recommendation system.

Key Words

Android • Node.js • TMDb API • Artificial Intelligence • Groq API • LLaMA 3.3 • Google ML Kit • OSMDroid • Computer Vision • Geolocation • Client/Server Architecture

Liste des Abréviations

API	Application Programming Interface
AI/IA	Artificial Intelligence / Intelligence Artificielle
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
MVC	Model-View-Controller
ORM	Object-Relational Mapping
OSM	OpenStreetMap
REST	Representational State Transfer
SDK	Software Development Kit
TMDB	The Movie Database
UML	Unified Modeling Language
URL	Uniform Resource Locator
UI/UX	User Interface / User Experience

Introduction Générale

Le secteur de l'industrie cinématographique connaît une transformation profonde sous l'impulsion du numérique. Les plateformes de streaming et les applications mobiles ont radicalement modifié les habitudes des cinéphiles, qui recherchent désormais une expérience immersive, personnalisée et accessible en tout lieu.

Dans ce contexte, ce projet de fin d'études vise à concevoir et développer CinéApp, un écosystème mobile complet qui transcende les simples applications de catalogue filmique. En s'appuyant sur les dernières avancées en matière d'intelligence artificielle, de vision par ordinateur et de géolocalisation, CinéApp ambitionne d'offrir une expérience cinématographique enrichie et unique.

La motivation principale de ce projet réside dans la convergence de plusieurs technologies : l'accès à une base de données cinématographique mondiale via l'API TMDb, l'intégration de l'intelligence artificielle générative via le modèle LLaMA 3.3 de Groq pour des analyses approfondies, et l'exploitation des capteurs Android pour des fonctionnalités immersives (caméra, GPS).

Problématique

Comment concevoir une application mobile Android qui intègre de manière cohérente les données cinématographiques en temps réel, l'intelligence artificielle, la vision par ordinateur et la géolocalisation, tout en offrant une expérience utilisateur fluide et sécurisée ?

Structure du Rapport

Ce rapport est organisé en quatre chapitres principaux, précédés des éléments préliminaires et suivis d'une conclusion générale :

- Chapitre 1 – Contexte Général du Projet : présentation du cadre du projet, analyse de la problématique, solution proposée et méthodologie de travail.
- Chapitre 2 – Analyse et Conception : étude fonctionnelle, analyse des besoins, règles de gestion et diagrammes UML.
- Chapitre 3 – Étude Technique : architecture choisie, technologies utilisées et environnement de travail.
- Chapitre 4 – Mise en Œuvre : présentation de l'application développée et pipeline DevOps.

Chapitre 1 : Contexte Général du Projet

Introduction

Ce premier chapitre pose les fondations de notre projet en définissant son contexte, en identifiant la problématique à laquelle il répond et en présentant la solution retenue ainsi que la méthodologie de travail adoptée.

I – Problématique

Les applications cinématographiques existantes souffrent de plusieurs lacunes majeures. D'une part, elles se limitent généralement à un simple catalogue de films sans valeur ajoutée personnalisée. D'autre part, elles ne tirent pas parti des capacités avancées des smartphones modernes (IA embarquée, caméra, GPS).

Plus précisément, les défis identifiés sont :

- L'absence d'analyse intelligente des films accessible aux utilisateurs ordinaires.
- Le manque d'interactivité basée sur la vision par ordinateur (filtres, avatars).
- L'absence de géolocalisation des salles de cinéma à proximité.
- Une expérience d'authentification souvent peu sécurisée ou ergonomique.

II – Solution Proposée

CinéApp répond à ces problématiques en proposant un écosystème mobile complet articulé autour de cinq axes fonctionnels avancés (A1 à A5), construits sur une architecture robuste client/serveur :

- A1 – Authentification sécurisée (Login/Register) via backend Node.js/Express.
- A2 – Catalogue cinématographique enrichi via l'API TMDB avec lecture de trailers YouTube.
- A3 – Analyse IA des films via le modèle LLaMA 3.3 (Groq API).
- A4 – Filtres et détection de visages via Google ML Kit (vision par ordinateur).
- A5 – Géolocalisation des cinémas proches via OSMDroid/OpenStreetMap.

III – Méthodologie de Travail & Planification

La méthodologie adoptée est agile et itérative, inspirée du framework Scrum. Le projet a été découpé en sprints de deux semaines, permettant des livraisons incrémentales et une adaptation continue aux retours obtenus.

Les grandes phases du projet sont :

- Phase 1 – Analyse et spécification des besoins (Semaines 1–2)
- Phase 2 – Conception architecturale et UML (Semaines 3–4)

- Phase 3 – Développement Backend (Node.js/Express) (Semaines 5–6)
- Phase 4 – Développement Frontend Android (Semaines 7–10)
- Phase 5 – Intégration des APIs tierces et fonctionnalités avancées (Semaines 11–13)
- Phase 6 – Tests, corrections et déploiement (Semaines 14–15)

Conclusion

Ce chapitre a permis de cadrer le projet CinéApp en définissant clairement la problématique, la solution envisagée et la méthodologie de développement. Le chapitre suivant approfondira l'analyse fonctionnelle et la conception du système.

Chapitre 2 : Analyse et Conception

Introduction

Ce chapitre détaille l'analyse fonctionnelle du système CinéApp et présente les choix de conception retenus. Il couvre l'identification des acteurs et des cas d'utilisation, l'analyse des besoins fonctionnels et non fonctionnels, les règles de gestion métier, et les principaux diagrammes UML.

I – Étude Fonctionnelle

I-1 – Analyse des Besoins Fonctionnels

Les besoins fonctionnels ont été identifiés à travers des interviews avec des utilisateurs cibles et l'analyse des applications concurrentes. Ils se structurent autour des modules suivants :

- Module Authentification : inscription, connexion, gestion du profil et photo de profil.
- Module Catalogue : affichage des films populaires, recherche, détails du film, lecture de trailers.
- Module IA : analyse intelligente d'un film sélectionné via le modèle LLaMA 3.3.
- Module Vision : détection de visage en temps réel, application de filtres/avatars.
- Module Géolocalisation : affichage de la carte, localisation GPS, cinémas proches.

I-2 – Analyse des Besoins Non Fonctionnels

- Performance : temps de réponse des requêtes réseau inférieur à 3 secondes.
- Sécurité : authentification sécurisée avec hachage des mots de passe (bcrypt) et tokens JWT.
- Ergonomie : interface intuitive respectant les guidelines Material Design.
- Maintenabilité : architecture modulaire et code documenté.
- Compatibilité : Android API 21+ (Android 5.0 Lollipop et supérieur).

II – Étude Conceptuelle

II-1 – Règles de Gestion

- Un utilisateur doit s'authentifier pour accéder aux fonctionnalités de l'application.
- Un film peut être marqué comme favori et ajouté à une liste personnalisée.
- L'analyse IA est limitée à un film à la fois et nécessite une connexion internet active.
- La géolocalisation nécessite l'autorisation GPS de l'utilisateur.
- Les filtres de vision par ordinateur s'appliquent en temps réel via le flux caméra.
- La photo de profil est capturée via l'appareil photo et stockée localement.

II-2 – Diagrammes UML

La conception du système s'appuie sur les diagrammes UML suivants :

- Diagramme de cas d'utilisation : représente les interactions entre les acteurs (Utilisateur, Administrateur) et le système.
- Diagramme de classes : modélise les entités principales (Utilisateur, Film, Favori, Analyse) et leurs relations.
- Diagramme de séquence : illustre les flux de communication pour les scénarios clés (authentification, requête TMDB, analyse IA).
- Diagramme d'activité : décrit le flux de navigation de l'application.

Conclusion

L'analyse fonctionnelle et la conception réalisées dans ce chapitre constituent le référentiel de développement du projet. Les diagrammes UML produits serviront de guide tout au long des phases d'implémentation. Le chapitre suivant détaillera les choix techniques qui permettront de concrétiser cette architecture.

Chapitre 3 : Étude Technique

Introduction

Ce chapitre présente l'architecture technique retenue pour CinéApp ainsi que l'ensemble des technologies et outils utilisés lors du développement. Il justifie les choix effectués et décrit l'environnement de travail mis en place.

I – Architecture Choisie

L'application CinéApp repose sur une architecture client/serveur à deux niveaux, enrichie par l'intégration d'APIs tierces :

- Couche Client (Android) : interface utilisateur native Java, gestion de l'état local et appels réseau asynchrones via Volley.
- Couche Serveur (Node.js) : API REST sécurisée gérant l'authentification et les données utilisateurs.
- Couche Services Tiers : TMDb API (données cinématographiques), Groq API (IA générative), Google ML Kit (vision), OSMDroid (cartographie).

Cette architecture garantit une séparation claire des responsabilités, une évolutivité aisée et une maintenance simplifiée.

II – Technologies Utilisées

Frontend Android

- Langage : Java (Android natif)
- SDK : Android SDK (API 21+, compilé en API 34)
- Requêtes réseau : Volley (gestion du cache, file de requêtes)
- Chargement d'images : Glide (cache, transformations, GIF)
- Vision par ordinateur : Google ML Kit (détection de visages, landmarks)
- Cartographie : OSMDroid (OpenStreetMap, marqueurs, polygones)
- Lecture vidéo : YouTube Android Player API
- Interface utilisateur : Material Design Components

Backend Node.js

- Runtime : Node.js (v18 LTS)
- Framework web : Express.js
- Authentification : JWT (JSON Web Tokens) + bcrypt (hachage mots de passe)
- Base de données : MongoDB avec Mongoose ODM
- Validation des données : Joi
- Middleware : CORS, helmet (sécurité HTTP), morgan (logs)

APIs Tierces

- TMDB API v3 : données de films, images haute résolution, bandes-annonces, métadonnées.
- Groq API (LLaMA 3.3 70B) : génération de texte, analyse sémantique, recommandations.

III – Environnement de Travail

- IDE Android : Android Studio (Hedgehog 2023.1.1)
- IDE Backend : Visual Studio Code
- Gestion de version : Git / GitHub
- Test d'API : Postman
- Émulateur : Android Virtual Device (Pixel 6, API 33)
- Système d'exploitation de développement : Windows 11 / macOS

Conclusion

Les choix techniques effectués dans ce chapitre répondent aux exigences fonctionnelles et non fonctionnelles identifiées dans le chapitre précédent. L'utilisation de technologies éprouvées (Java natif, Node.js/Express) combinée à des APIs de pointe (Groq, ML Kit) permet de construire une application robuste, performante et innovante. Le chapitre suivant présentera la mise en œuvre concrète de ces choix.

Chapitre 4 : Mise en Œuvre

Introduction

Ce dernier chapitre présente les résultats concrets du développement de CinéApp. Il décrit les principales interfaces réalisées, les fonctionnalités implémentées et le pipeline DevOps mis en place pour assurer la qualité et la livraison continue du projet.

I – Présentation de l'Application

Authentification (A1)

Le module d'authentification développé en Node.js/Express offre :

- Écran d'inscription avec validation des champs (email, mot de passe, pseudo).
- Connexion sécurisée avec génération d'un token JWT valide 7 jours.
- Capture et stockage de la photo de profil via l'appareil photo Android.
- Persistance de la session via SharedPreferences.

Catalogue Cinématographique (A2)

L'intégration de l'API TMDb permet d'afficher :

- La liste des films populaires, tendances et à venir, avec pagination infinie.
- La fiche détaillée de chaque film (synopsis, note, casting, genres, date de sortie).
- La lecture des bandes-annonces YouTube intégrée directement dans l'application.
- Un moteur de recherche en temps réel avec suggestions.

Analyse IA des Films (A3)

L'intégration de l'API Groq avec le modèle LLaMA 3.3 70B permet :

- L'analyse thématique et sémantique d'un film sélectionné.
- La génération de critiques intelligentes et de recommandations personnalisées.
- Un chatbot contextuel permettant de poser des questions sur le film.

Vision par Ordinateur – Filtres (A4)

Google ML Kit est utilisé pour :

- La détection en temps réel des visages via le flux caméra.
- L'identification des points clés du visage (yeux, nez, bouche, contour).
- L'application de filtres dynamiques et d'overlays d'avatars superposés au visage.
- La capture de photo avec le filtre appliqué, sauvegardable en galerie.

Géolocalisation des Cinémas (A5)

Le module de géolocalisation basé sur OSMDroid offre :

- L'affichage d'une carte interactive OpenStreetMap dans l'application.

- La localisation GPS de l'utilisateur en temps réel.
- La recherche et l'affichage des salles de cinéma dans un rayon configurable.
- L'affichage d'informations détaillées sur chaque cinéma (nom, adresse, horaires).

II – DevOps

Un pipeline DevOps léger a été mis en place pour garantir la qualité du code et faciliter les déploiements :

- Contrôle de version : GitHub avec branches feature/develop/main et Pull Requests.
- Intégration continue : GitHub Actions pour les builds automatiques Android (APK debug).
- Tests automatisés : JUnit pour les tests unitaires Android, Jest pour le backend Node.js.
- Analyse de code : SonarLint intégré dans Android Studio pour la détection de code smells.
- Déploiement backend : Node.js sur serveur cloud (Render.com / Railway).
- Distribution APK : Firebase App Distribution pour les tests bêta.

Conclusion

La mise en œuvre de CinéApp a permis de concrétiser l'ensemble des fonctionnalités définies lors des phases d'analyse et de conception. L'application développée répond aux objectifs fixés en termes de fonctionnalités, de performance et d'expérience utilisateur. Le pipeline DevOps mis en place garantit la maintenabilité et l'évolutivité du projet.

Conclusion Générale & Perspectives

Ce projet de fin d'études nous a permis de concevoir et développer CinéApp, un écosystème mobile Android complet qui illustre la convergence de plusieurs domaines technologiques de pointe : développement mobile natif, architecture client/serveur, intelligence artificielle générative, vision par ordinateur et géolocalisation.

Sur le plan technique, nous avons maîtrisé l'intégration d'APIs tierces complexes (TMDB, Groq), l'utilisation de bibliothèques Android avancées (Volley, Glide, ML Kit, OSMDroid) et la mise en place d'un backend sécurisé avec Node.js/Express. Cette expérience nous a dotés d'une vision globale du cycle de développement logiciel, de la conception UML jusqu'au déploiement DevOps.

Sur le plan humain, ce projet nous a appris à travailler en équipe, à gérer les délais, à résoudre des problèmes complexes et à nous adapter aux changements technologiques rapides du secteur.

Perspectives d'Évolution

Plusieurs améliorations et extensions sont envisageables pour faire évoluer CinéApp :

- Système de recommandation avancé : intégration d'un moteur de recommandation collaboratif basé sur les préférences de l'ensemble des utilisateurs.
- Mode hors-ligne : mise en cache intelligente des données pour une utilisation sans connexion.
- Fonctionnalités sociales : partage de critiques, listes collaboratives et suivi d'amis.
- Réalité augmentée : enrichissement des filtres de vision par des éléments AR (décors de films, personnages 3D).
- Version iOS : portage de l'application sur la plateforme Apple via React Native ou Flutter.
- Monétisation : intégration d'un système d'abonnement et de réservation de billets en ligne.

Références

- [1] Android Developers. (2024). Android Developer Documentation.
<https://developer.android.com/docs>

- [2] The Movie Database (TMDB). (2024). TMDB API Reference.
<https://developer.themoviedb.org/docs>

- [3] Groq. (2024). Groq API Documentation – LLaMA 3.3. <https://console.groq.com/docs>

- [4] Google. (2024). ML Kit for Android – Face Detection.
<https://developers.google.com/ml-kit/vision/face-detection>

- [5] OSMDroid. (2024). OSMDroid – OpenStreetMap Android API.
<https://github.com/osmdroid/osmdroid>

- [6] Express.js. (2024). Express.js Documentation. <https://expressjs.com>

- [7] MongoDB. (2024). MongoDB Documentation. <https://www.mongodb.com/docs>

- [8] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

- [9] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.

- [10] Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.

Annexes

Annexe A – Extrait du Code Source Backend

Exemple de route d'authentification Node.js/Express :

```
POST /api/auth/login → Validation email/password → bcrypt.compare() → JWT.sign()  
→ Response 200 {token}
```

Annexe B – Configuration Gradle (Android)

Les dépendances principales déclarées dans build.gradle :

```
implementation 'com.android.volley:volley:1.2.1'  
implementation 'com.github.bumptech.glide:glide:4.16.0'  
implementation 'com.google.mlkit:face-detection:16.1.5'  
implementation 'org.osmdroid:osmdroid-android:6.1.18'  
implementation 'com.google.android.gms:play-services-location:21.0.1'
```